COMPSCI 713: AI Fundamentals

Lecture 7: MYCIN

A Backward-Chaining Production-Rule Expert System for Medical Diagnosis

Instructor: Xinyu Zhang
https://zhangxinyu-xyz.github.io/

School of Computer Science, University of Auckland

S1 2026

Overview

- We will discuss MYCIN, a breakthrough system developed by Ted Shortliffe at Stanford in the 1970s.
- It applied AI to medical diagnosis and treatment decision-making in the domain of bacterial infections.
- It was significant for its ambition: one of the earliest **expert systems**.
- It aimed to emulate a specialist in a narrow problem area.
- It was also significant for its technical approach to knowledge representation and inference:
  - separating knowledge (data) from inference (code),
  - explanation capability,
  - easier knowledge editing,
  - and a leading-edge search strategy.

A MYCIN Production Rule

- This rule has three predicates (Boolean conditions that determine whether it should *fire*); this is the rule's **premise**.
- Each predicate compares a patient-related data field with a qualitative value (e.g., `[stain of the organism] = gram negative`).
- Human expertise is still required, for example to determine that the organism morphology is a *rod*.
- The rule produces a new fact about the organism identity; this fulfills a goal or sub-goal.
- This is **symbolic reasoning**: working with concepts rather than numeric formulas.

Example rule

```
IF the stain of the organism is gram negative
   AND the morphology of the organism is rod
      AND the aerobicity of the organism is anaerobic
         THEN there is suggestive evidence (0.6) that
            the identity of the organism is Bacteroides.
```

Under the Covers: Implementation with LISP

- The rule is implemented internally in **LISP** (the List Processing language).
- The user interface presents a more readable translation for explanation, but it remains close to the implementation.
- Over the life of the system, the number of rules grew to around 200.

Internal representation (illustrative)

```
PREMISE: ($AND (SAME CNTXT GRAM GRAMNEG)
               (SAME CNTXT MORPH ROD)
               (SAME CNTXT AIR ANAEROBIC))
ACTION:  (CONCLUDE CNTXT IDENT BACTEROIDES TALLY .6)
```

CF(conclusion) = CF(premise) × CF(rule)

- IF A ⇒ B. CF(premise) = CF(A)
- IF A AND B AND C ⇒ D. CF(premise) = min(CF(A), CF(B), CF(C)); CF(D) = CF(premise) × CF(rule)

LISP

- The **List Processing** language is centered on lists containing symbols, numbers, and/or nested lists.
- In classic LISP, code and data share the same list-based representation.
- It uses prefix notation; for example, `(+ 1 2)` yields 3.

Example

```
(defun factorial (N)
  "Compute the factorial of N."
  (if (= N 1)
      1
      (* N (factorial (- N 1)))))
```

LISP was developed in the early 1960s, and Common Lisp is still available today.

Making and Evaluating Lists in LISP

Examples

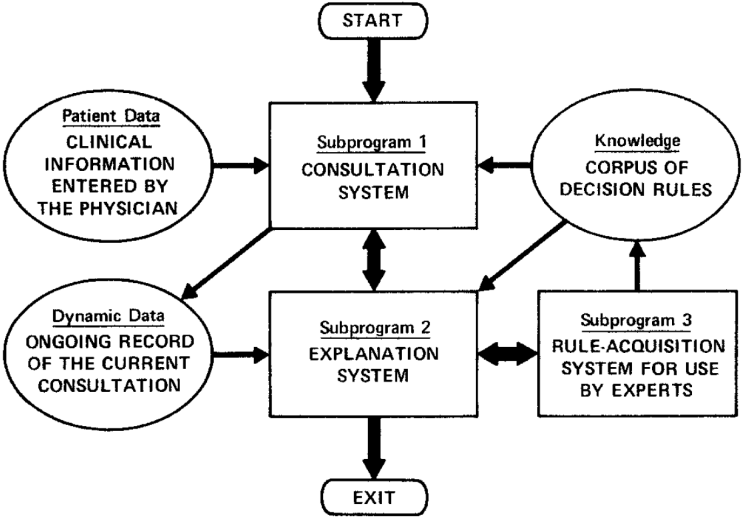
```
(cons 'a (cons 'b '())) ==> (A B)
or
(list 'a 'b) ==> (A B)

(setq form1 '(+ 1 2 3))
(eval form1) ==> 6
```

- *cons* = adds an element to the front of a list; *list* = build list; *setq* = *set* + *quote*, assign values to variables; *eval* = evaluation.
- $'$ = quote (data, not evaluation)
- $'()$ = the empty list
- *form*1 = (+ 1 2 3); (+ 1 2 3) $\rightarrow$ 6

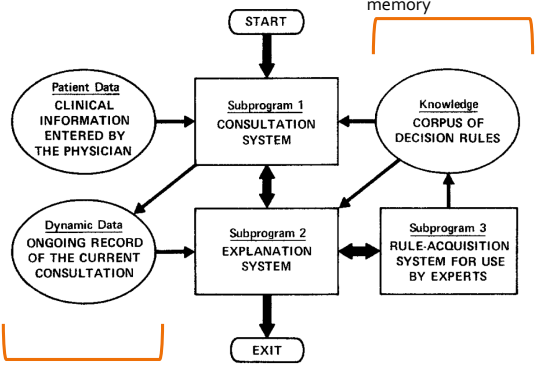
MYCIN uses the fact that a program can **construct lists** and then **evaluate** them, allowing the system to accept new production rules and execute them during a consultation.

MYCIN Architecture



MYCIN Architecture

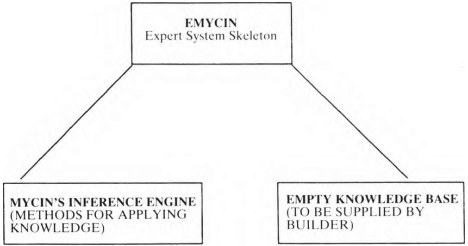
Knowledge base – Persistent (with periodic updates); like long-term memory



Working memory – Different for each case; updates from user input and firing of production rules; like short-term memory

Re-usable Architecture

- The **Consultation**, **Explanation**, and **Rule-Acquisition** systems are NOT specific to MYCIN's medical domain.
- MYCIN effectively provided the first expert-system **shell**.
  - We can replace the domain knowledge and use the same software architecture to solve a different problem.
  - It was pretty revolutionary – normally you'd write a computer program to solve a specific problem
- **E-MYCIN** ("empty MYCIN") was later used to implement other expert systems.

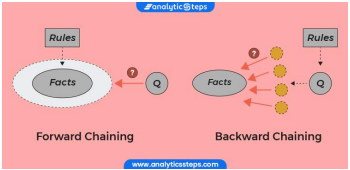


Inference Engine (total control system)

- The inference engine is the **reasoning** part of the system, as opposed to the **knowledge** itself.
- MYCIN refers to `MONITOR` and `FINDOUT` functions.
  - `MONITOR` scans for rules that can be used.
  - If a rule's premise is false, that rule is discarded.
    - If any condition in a conjunction (joined by AND parts) is false, the entire premise is false.
  - If some needed premise value is unknown, the inference engine invokes `FINDOUT`.
  - The missing value may come from a database (e.g., a lab result) or from asking the user.
- Recursive use of `MONITOR` and `FINDOUT` can follow a line of reasoning until it is falsified or a goal is confirmed.
  - Or maybe the user would rather pursue another line of reasoning for a while because of the cost of answering a question.

Forward and Backward Chaining (how to reason)

- **Forward chaining** uses modus ponens:
  - if premise p is true and $p \Rightarrow q$, then q is true;
  - p may itself be a conjunction of multiple conditions;
  - q may become the premise (or part of the premise) of another rule, such as $q \Rightarrow r$.
  - Forward chaining is useful when there is a fixed set of data collected before reasoning begins.
- **Backward chaining** seeks to prove a hypothesis.
  - We start from a desired goal (for example, identifying the organism causing an infection).
  - We then try to confirm the premise required for that goal.
  - If some premise value is unknown, finding it out becomes a **sub-goal**.
  - The sub-goal may itself be answered by firing another rule, or so on recursively.



Examples

Forward chaining (data-driven)

Fact: The patient has a fever.

Rule: If a patient has a fever and a rash, they might have measles.

Starting with the known fact (fever), the system checks for other symptoms (rash). If the patient also has a rash, the system infers the possibility of measles.

Backward chaining (goal-driven)

Goal: Determine why the network is down.

Rule: If the router is malfunctioning, the network will be down.

The system starts with the goal (network down) and works backward to check if the router is malfunctioning, verifying the necessary conditions to confirm the hypothesis.

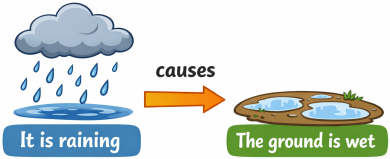
Reference example source: `https://www.geeksforgeeks.org/forward-chaining-and-backward-chaining-inference-in-rule-based-systems/`

Forward vs Backward Chaining and Logical Conditions

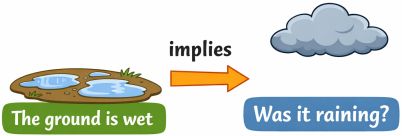
- A production rule of the form: IF A THEN B.
- Logically:
  - A is a **sufficient condition** for B
  - B is a **necessary condition** for A
- **Forward Chaining**:
  - Reasoning direction: $A \rightarrow B$
  - Starts from known facts and derives conclusions
  - Uses sufficient conditions to infer new facts
- **Backward Chaining**:
  - Reasoning direction: $B \rightarrow A$
  - Starts from a goal and tries to prove it
  - Checks whether necessary conditions are satisfied
- Important note:
  - IF A THEN B does **not** mean A is the only cause of B
  - Backward chaining identifies **possible** necessary conditions, not unique ones

Example: Sufficient vs Necessary in Chaining

- Consider the rule:
  - IF it is raining THEN the ground is wet
- **Forward Chaining (Sufficient Condition)**:
  - It is raining $\Rightarrow$ the ground is wet
  - Raining is sufficient to conclude wet ground
- **Backward Chaining (Necessary Condition)**:
  - The ground is wet $\Rightarrow$ was it raining?
  - Check if raining (a possible necessary condition) holds
- Important observation:
  - Wet ground does not imply rain was the only cause
  - Other causes may exist (e.g., sprinklers)



The ground is wet because it is raining.



If the ground is wet, it was likely raining.

Backward Chaining in MYCIN

- MYCIN begins a consultation with a fixed set of heuristically chosen questions.
- These questions often cover:
  - information usually needed to reach an answer, and
  - information that can rule out rarer conclusions.
- Then MYCIN would go into backward chaining mode to try to prove a specific goal.
  - It uses confidence factors as a guide.
  - It starts with the goal that can potentially be proved with the greatest confidence.
    - If system has multiple conclusions: 1) Bacteroides (CF=0.8); 2) E. coli (CF=0.6); A rare bacteria (CF=0.2). Start from Bacteroides.
  - It abandons a line of reasoning when confidence falls below 0.2.
    - Chaining line: Conclusion $\rightarrow$ Condition 1 $\rightarrow$ Condition 2 $\rightarrow$ Condition 3
    - CF(prime) $= 0.5$; CF(rule) $= 0.6$
    - CF(Condition 1) $= 0.5 \times 0.6 = 0.3$
    - CF(Condition 2) $=$ CF(Condition 1)$\times$ CF(rule) $= 0.3 \times 0.6 = 0.18$ (abandon)

Explanation

- Explanation was considered essential if clinicians were to trust the recommendations.
  - The healthcare professional remains responsible for the final decision, so they need enough transparency to "*own*" it.
- MYCIN supported `WHY` and `HOW` queries, with optional depth parameters (or repeating of the command).
  - **`WHY` queries (goal-driven explanation)**
    - A `WHY` response states the current sub-goal in the context of its production rule.
    - This explains **"why" the system is asking for a particular premise value**.
    - Further `WHY` queries (or specifying depth, `WHY-3`) explains further steps in the backward chain to the high-level goal.
    - Closely related to **backward chaining**.
    - E.g., User: "WHY are you asking if the organism is anaerobic?"; System: "Because I am trying to determine whether it is Bacteroides, and for that I need to know if it is anaerobic."
  - **`HOW` queries (data-driven explanation)**
    - `HOW` queries explain **what premises led the system to conclude** that a fact was true.
    - Follow the reasoning path from known facts to conclusions.
    - Similar to **forward chaining**, but mainly an *explanation of inference*.
    - E.g., User: "HOW did you conclude it is Bacteroides?"; System: "Because: - It is gram negative - It is rod-shaped - It is anaerobic; therefore Bacteroides (CF=0.6)"

Example: WHY vs HOW and Chaining

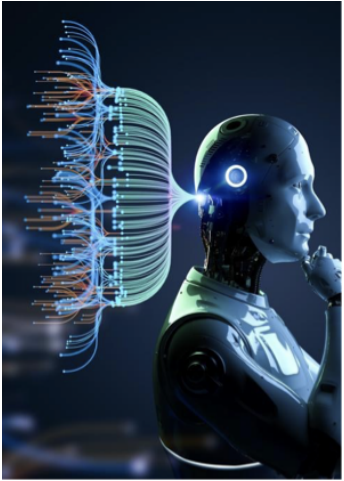
- **Key distinction**
  - `WHY`: explains *why a question is being asked* (backward reasoning).
  - `HOW`: explains *how a conclusion was reached* (forward-like explanation).
- Example: Consider the rule: **IF** it is raining **THEN** the ground is wet.
- `WHY` **(Backward Chaining Style)**
  - Goal: determine whether the ground is wet.
  - System asks: "Is it raining?"
  - Explanation: "I need to know if it is raining to conclude that the ground is wet."
- `HOW` **(Forward Explanation Style)**
  - Known fact: it is raining.
  - Rule applied: IF raining THEN ground is wet.
  - Explanation: "Since it is raining, the system concludes that the ground is wet."
- **Interpretation**
  - `WHY`: traces reasoning from goal to required facts.
  - `HOW`: traces reasoning from facts to conclusion.

Evaluation

- MYCIN was evaluated using a panel of experts and a set of real-world cases.
  - Abstracts were prepared for 15 hospital patients with positive blood cultures.
  - 10 physicians specializing in infectious disease (5 from Stanford and 5 from elsewhere) answered for each case:
    - organism significance (i.e. is it a pathogen requiring treatment?),
    - organism identity,
    - and treatment.
  - A very high level of agreement was found: MYCIN agreed with the experts about as well as they agreed with one another.
- **Agreement with a human expert panel** is one way to evaluate such a system.
- Another is to **measure whether clinicians using MYCIN achieve better health outcomes than those who do not**, after controlling for other factors.
- MYCIN never reached that stage, such as Medical liability issues, Legal risks, Ethical concerns, Lack of full trust from doctors in machine recommendations, etc.

Why "Expert Systems"?

- What good is it to have an AI system that emulates an expert?
  - It can help a human expert recall relevant knowledge for the current case.
    - Humans are prone to "lapses"; computers are consistent and do not get tired.
    - A computer can maintain a much larger working memory and search a large knowledge base quickly. Humans are limited to about 7 "chunks" at any one time, and we need a whiteboard or such to solve more involved problems.
  - Sometimes an expert is unavailable (e.g., frontline care, disasters, battlefields).
- Even so, expert systems did not become mainstream in medicine.
  - The healthcare professional still carries responsibility.
  - In practice, this is often framed as a **decision support system** to provide "help".
- It remains very hard to encode **all** applicable expert knowledge.



Knowledge Acquisition "Bottleneck"

- Not practical to expect domain experts to enter all of their knowledge directly into the system.
  - MYCIN's acquisition tools mostly supported expert review of rules, or limited tweaking among pre-specified concepts.
- Knowledge is always changing.
  - If you ran MYCIN today, you would effectively be practicing 1970s-era medicine.
  - There are new drugs, new clinical evidence, and even new problems such as antibiotic-resistant organisms.
  - Even with a perfect knowledge base, constant maintenance would still be required.
- In reality, the knowledge base was never perfect.
  - In the 1970s, people often thought the solution was simply to specify more rules.
  - Shortliffe already noted in 1974 that the rules numbered around 200 and that many more were anticipated.
  - Large production-rule systems are also very hard to debug.

Where Have Things Gone Since?

- Soon after the Web emerged in the early 1990s, there was strong interest in creating a **Semantic Web**.
  - **RDF** (Resource Description Framework) specifies metadata.
  - **OWL** (Web Ontology Language) specifies relationships for reasoning.
  - **Wikidata** provides an explicit structured-knowledge counterpart to Wikipedia.
- **IBM Watson**, especially from 2011 onward, showed the potential of systems that could read natural language and answer questions.
- **GPT-style models** provide impressive breadth and fluency, but their mechanism is quite different from explicit symbolic reasoning.
  - As a result, they can still make striking logic errors, especially when multi-step reasoning is required.

Practical Applications Now for Production Rules?

- `Drools` (server-side Java) and `Nools` (JavaScript / Node.js) are examples of business-rules engines. They often offer better software engineering practice than hard-coding complex logic.
- They are useful for policies that change frequently, or differ by region.
- Examples include:
  - Common domains include finance and healthcare, especially health insurance.
  - Insurance entitlements that differ by plan,
  - Rules dictated by government policy (e.g., Medicare),
  - and region-specific employment rules.
- In the New Zealand / Australia context, superannuation and leave entitlements may differ across jurisdictions.
- Backward chaining can also support "softer" recommendation tasks, such as suggesting a suitable insurance policy, home loan, or investment option for a customer's needs.

Original slide included this external link: `https://www.nected.ai/blog/top-10-business-rules-engine`

Conclusion

- MYCIN was groundbreaking.
  - It assembled and operationalized rules to perform a credible version of a task normally requiring highly trained professionals.
  - It demonstrated "sleuth"-like hypothesis testing through backward chaining.
  - It provided a reusable shell onto which very different knowledge bases could be authored.
- MYCIN was valuable not only for what it achieved, but also for the limitations it revealed.
  - Over time, researchers realized that expert systems were not a practical route to advanced professional intelligence at scale.
    - This was not the path to dramatic general AI.
    - But it remains a good approach for automating tedious, complex policies and for sorting through many structured product options.
  - It also pushed people to think carefully about responsibility in human–AI systems.

In Coming Lectures...

A hint of what comes next

Notice the confidence factor in MYCIN: *"suggestive evidence (0.6)"*.

- This points toward the next phase of AI, which is more concerned with probability, uncertainty, and approximation.

References I

Shortliffe, E., Davis, R., Axline, S., Buchanan, B., Green, C., & Cohen, S. (1975).
Computer-based consultations in clinical therapeutics: Explanation and rule-acquisition capabilities of the MYCIN system.
Computers and Biomedical Research, 8, 303–320.

Yu, V., Buchanan, B., Shortliffe, E., Wraith, S., Davis, R., Scott, A., & Cohen, S. (1979).
Evaluating the performance of a computer-based consultant.
Computer Programs in Biomedicine, 9, 95–102.